

MediaFlow Jobs Platform — Implementation Specification

Share this brief with the developer/implementation agent

Upgrade the current MediaFlow application into a production-oriented, LinkedIn/Unstop-style software careers platform. Preserve the existing media-processing workflow, but add a complete authenticated jobs marketplace and candidate application pipeline.

Product outcome

Users should be able to:

1. Visit a polished jobs discovery homepage.
2. Sign in with Google or Microsoft.
3. Search and filter live software opportunities.
4. Open a complete job detail page.
5. Save jobs to a personal shortlist.
6. Apply through the platform or save an external application URL.
7. Move applications through a personal pipeline:
 - Saved
 - Applied
 - Screening
 - Interview
 - Offer
 - Rejected
8. Upload a resume or portfolio media file through the existing MediaFlow processing pipeline.
9. Track resume/portfolio processing status.
10. View all personal data from an authenticated dashboard.
11. Delete personal data and disconnect the account.
12. Receive secure, deduplicated notifications for application updates.

Important production rule

Do **not** store real user accounts, OAuth tokens, resumes, or application records in a local JSON file, browser localStorage, `/tmp`, or a Vercel function filesystem. Those locations are temporary or user-specific and are not reliable persistence.

Use:

- **PostgreSQL** for users, jobs, applications, saved jobs, profiles, audit events, and notification state.
- **Object storage** for resumes, portfolios, and media files.
- **Signed short-lived URLs** for upload and download.
- **Managed authentication** for Google and Microsoft OAuth.
- **Encrypted secrets** in Vercel project environment variables or a secrets manager.

A suitable low-setup option is Supabase Auth + Supabase Postgres + Supabase Storage. A production alternative is Auth0/Clerk for identity, Neon/RDS for Postgres, and S3 for files.

Job categories

The marketplace must support category and skill taxonomy for at least:

- Software Engineer
- SDE / SDE I / SDE II / SDE III
- Backend Engineer
- Frontend Engineer
- Full-Stack Engineer
- Platform Engineer
- DevOps Engineer
- Site Reliability Engineer
- Cloud Engineer
- Data Analyst
- Data Engineer
- Data Scientist
- Machine Learning Engineer
- AI Engineer

- Applied Scientist
- MLOps Engineer
- Security Engineer
- QA / Test Automation Engineer
- Mobile Engineer
- Embedded Engineer
- Product Manager
- Technical Writer
- UI/UX Designer
- Internships
- Freelance / Contract
- Hackathons
- Hiring Challenges
- Fellowships

Every job should support tags such as Python, Java, JavaScript, TypeScript, React, Node.js, Go, Rust, AWS, Azure, GCP, SQL, Spark, Kafka, Docker, Kubernetes, Terraform, LLM, RAG, computer vision, NLP, analytics, and system design.

Authentication requirements

Implement OAuth login with both providers:

Google

Required OAuth configuration:

- Google OAuth Client ID
- Google OAuth Client Secret
- Authorized redirect URI for local development
- Authorized redirect URI for production
- Consent screen configured for the application domain

Microsoft

Required Entra ID / Microsoft OAuth configuration:

- Microsoft Application / Client ID
- Client Secret or certificate
- Tenant setting: `common` for personal and organizational accounts, or a specific tenant when required
- Web redirect URI for local and production environments

Required authentication behavior

- Email/password must not be required when OAuth is enabled.
- Verify the identity provider token server-side.
- Store only provider subject ID, email, name, avatar URL, and consent metadata.
- Never store provider access tokens unless a specific integration needs them.
- If tokens are required, encrypt them and store their expiry/refresh state server-side.
- Use secure, HTTP-only, SameSite cookies for sessions.
- Rotate session identifiers after login.
- Protect state and nonce values against CSRF and replay attacks.
- Add rate limits to login, callback, passwordless, and sensitive account endpoints.
- Provide logout, account deletion, and active-session revocation.

Core data model

```
users (  
  id uuid primary key,  
  email text unique not null,  
  display_name text,  
  avatar_url text,  
  headline text,  
  location text,  
  bio text,  
  profile_visibility text default 'private',  
  created_at timestampz not null,  
  updated_at timestampz not null  
)  
  
identities (  
  id uuid primary key,  
  user_id uuid references users(id) on delete cascade,  
  provider text not null, -- google or microsoft  
  provider_subject text not null,  
  provider_email text,  
  created_at timestampz not null,  
  unique(provider, provider_subject)  
)  
  
jobs (  
  id uuid primary key,  
  external_id text,  
  source text not null,  
  title text not null,  
  company text not null,  
  company_logo_url text,  
  description text not null,  
  apply_url text,  
  employment_type text,  
  experience_level text,  
  location text,  
  remote_type text,  
  salary_min numeric,  
  salary_max numeric,  
  salary_currency text,  
  category text not null,  
  skills jsonb default '[]',
```

```

    posted_at timestamptz,
    expires_at timestamptz,
    is_verified boolean default false,
    raw_source_payload jsonb,
    created_at timestamptz not null,
    updated_at timestamptz not null,
    unique(source, external_id)
)

saved_jobs (
    user_id uuid references users(id) on delete cascade,
    job_id uuid references jobs(id) on delete cascade,
    note text,
    created_at timestamptz not null,
    primary key(user_id, job_id)
)

applications (
    id uuid primary key,
    user_id uuid references users(id) on delete cascade,
    job_id uuid references jobs(id) on delete cascade,
    status text not null, -- SAVED/APPLIED/SCREENING/INTERVIEW/OFFER/REJECTI
    applied_at timestamptz,
    next_action text,
    next_action_at timestamptz,
    private_notes text,
    created_at timestamptz not null,
    updated_at timestamptz not null,
    unique(user_id, job_id)
)

files (
    id uuid primary key,
    user_id uuid references users(id) on delete cascade,
    application_id uuid references applications(id) on delete set null,
    object_key text not null,
    file_name text not null,
    content_type text not null,
    size_bytes bigint not null,
    checksum text,
    processing_status text not null, -- QUEUED/PROCESSING/COMPLETED/FAILED
    created_at timestamptz not null,
    updated_at timestamptz not null
)

```

```
notifications (  
  id uuid primary key,  
  user_id uuid references users(id) on delete cascade,  
  type text not null,  
  title text not null,  
  body text not null,  
  read_at timestamptz,  
  created_at timestamptz not null  
)  
  
audit_events (  
  id uuid primary key,  
  user_id uuid,  
  event_type text not null,  
  request_id text,  
  metadata jsonb,  
  created_at timestamptz not null  
)
```

API contract

All private endpoints require a valid authenticated session.

```
GET    /api/auth/providers
GET    /api/auth/login/google
GET    /api/auth/login/microsoft
GET    /api/auth/callback/google
GET    /api/auth/callback/microsoft
POST   /api/auth/logout
GET    /api/me
PATCH /api/me
DELETE /api/me

GET    /api/jobs?q=&category=&skill=&location=&remote=&employmentType=&pa
GET    /api/jobs/{jobId}
POST   /api/jobs/{jobId}/save
DELETE /api/jobs/{jobId}/save

GET    /api/me/applications?status=&page=&limit=
POST   /api/jobs/{jobId}/applications
PATCH /api/applications/{applicationId}
DELETE /api/applications/{applicationId}

GET    /api/me/files
POST   /api/me/files/presign
GET    /api/me/files/{fileId}
DELETE /api/me/files/{fileId}

GET    /api/me/notifications
POST   /api/me/notifications/{notificationId}/read
GET    /api/health
GET    /api/overview
```

Standard response format

```
{
  "data": {},
  "meta": {
    "requestId": "req_...",
    "page": 1,
    "limit": 20,
    "total": 120
  },
  "error": null
}
```

Errors should use a consistent shape:

```
{
  "data": null,
  "meta": {"requestId": "req_..."},
  "error": {
    "code": "AUTH_REQUIRED",
    "message": "Sign in to continue"
  }
}
```

Live jobs ingestion

The backend should support a scheduled or queue-based ingestion process:

1. Fetch jobs from approved public APIs or partner feeds.
2. Normalize titles, company, category, skills, location, and dates.
3. Deduplicate using `source + external_id`.
4. Reject malformed or unsafe external URLs.
5. Store raw source payload separately for audit/debugging.
6. Mark verified jobs only after source validation.
7. Expire jobs automatically after their closing date or source freshness limit.

8. Never claim that a job is live unless the source was recently fetched successfully.
9. Use pagination and server-side filters; do not load every job into the browser.
10. Add an admin-only moderation workflow for source quality and reported jobs.

The current public job radar can be retained as a source adapter, but it should feed the database instead of acting as the primary user data store.

User application pipeline

Every application transition must be authorized for the owning user and recorded in an audit event.

```
SAVED → APPLIED → SCREENING → INTERVIEW → OFFER
                                   \→ REJECTED
SAVED → REJECTED
```

Required behavior:

- Optimistic UI updates with server reconciliation.
- Idempotent save/apply requests.
- Duplicate application protection using `unique(user_id, job_id)`.
- User notes are private by default.
- Support a next-action date and reminder.
- Preserve history when the status changes.
- Do not expose private notes through public job APIs.
- Add pagination for the application board.

Frontend experience

Main navigation

- Discover
- Jobs
- Challenges

- Internships
- My applications
- My profile
- Notifications

Homepage

- Personalized or general job recommendations.
- Search with instant suggestions.
- Category chips for all software roles.
- Remote/location filters.
- Featured verified opportunities.
- Closing-soon and recently-posted sections.
- Clear sign-in CTA for unauthenticated users.

Job detail page

- Job title, company, verified badge, location, salary, employment type.
- Full description with readable typography.
- Skills and experience sections.
- Save job action.
- Apply action.
- Similar jobs.
- Report listing action.
- Shareable canonical URL.

Authenticated dashboard

- Profile completeness card.
- Saved jobs.
- Application Kanban board.
- Upcoming next actions.
- Resume/portfolio processing status.
- Notification center.

- Account and privacy settings.

Security and privacy

- Row-level security so a user can only access their own applications, files, notes, and notifications.
- Server-side authorization on every private route.
- Content-Type allowlist and file-size limits for uploads.
- Malware scanning for resumes and uploaded media before downstream processing.
- Private object storage; no public bucket objects.
- Short-lived signed download URLs.
- Redact tokens, resume contents, and private notes from logs.
- Encrypt database, object storage, and sensitive fields at rest.
- Add CSP, HSTS, secure cookies, CSRF protection, and rate limiting.
- Add account export and account deletion flows.
- Keep audit events without retaining unnecessary personal content.
- Publish a privacy policy and terms before real users are onboarded.

Environment variables

```
DATABASE_URL=
AUTH_SESSION_SECRET=
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
GOOGLE_REDIRECT_URI=
MICROSOFT_CLIENT_ID=
MICROSOFT_CLIENT_SECRET=
MICROSOFT_TENANT=common
MICROSOFT_REDIRECT_URI=
STORAGE_BUCKET=
STORAGE_REGION=
STORAGE_ACCESS_KEY_ID=
STORAGE_SECRET_ACCESS_KEY=
JOB_SIGNING_SECRET=
CRON_INGEST_SECRET=
APP_ORIGIN=https://your-domain.vercel.app
```

Never commit this file with real values. Add the values in Vercel Project Settings → Environment Variables and keep separate Development, Preview, and Production values.

Recommended implementation phases

Phase 1 — foundation

- Choose Supabase/Auth0/Clerk and PostgreSQL provider.
- Add database migrations.
- Add server-side session middleware.
- Add Google and Microsoft OAuth callbacks.
- Add `/api/me` and account logout.

Phase 2 — job marketplace

- Create normalized jobs schema.
- Import live sources through a worker/cron.
- Add search, filtering, pagination, job details, and deduplication.

- Add admin moderation and source freshness.

Phase 3 — personal pipeline

- Add saved jobs and applications.
- Add Kanban dashboard and status history.
- Add notes, next actions, reminders, and notifications.

Phase 4 — media integration

- Add authenticated presigned uploads.
- Link resumes/portfolio files to user/application records.
- Preserve the existing SQS/Lambda processing contract.
- Store file processing status in PostgreSQL and object metadata.

Phase 5 — production hardening

- Add row-level authorization tests.
- Add OAuth security tests.
- Add rate limits, malware scanning, audit logging, and monitoring.
- Add deployment environments and rollback.
- Run load tests for job search and application updates.

Definition of done

- Google login works in Preview and Production.
- Microsoft login works in Preview and Production.
- Sessions survive page refresh and expire securely.
- A new user is created once and provider identity is deduplicated.
- A user can search all supported software job categories.
- A user can save, apply, update, and delete their own applications.
- Another user cannot read or mutate those records.
- Job data survives Vercel cold starts and redeployments.
- Resume/portfolio upload is private, signed, size-limited, and processed asynchronously.

- Every private API response includes a request ID.
- CI covers auth, authorization, job deduplication, application transitions, and upload security.
- Production documentation states which providers, OAuth credentials, domains, and database are configured.

Information required before live OAuth implementation

Provide or configure these values in the deployment platform:

1. Production domain or Vercel domain to use.
2. Google OAuth Client ID and Client Secret.
3. Microsoft Entra Application ID and Client Secret/certificate.
4. Database provider and connection string.
5. Object storage provider and bucket details.
6. Whether users can apply externally only, or whether an internal application form is required.
7. Whether job ingestion should use only approved sources or also allow employer submissions.

Until those values are configured, the application can provide a complete UI and mocked/local auth flow, but it must not claim that live Google/Microsoft authentication or durable production user data is active.